Introduction in Spectroscopy and measurement of the Balmer series.

Kiran Suda, Lukas Mengis
13 October 2026

Abstract

The goal of this experiment is to analyse the spectroscopy lines of hydrogene and helium and compare it with the literature values. To achive this we used a spectrometer and compared the differents angles of the grating of the spectral lines. At the end by taking the average over all result we got (3.07 ± 0.01)e15 Hz for the Ryderberg frequency, which is deviates with approxiamtely 7%[1] of the literature value[2]. One hypothesis is that the due to the bad resolution the mesearments are not totally correct. We conclude therefore that significant deviates of the lesser visible spectral lines have to be taken in account.

1 Introduction

Spectroscopy is used to spatialy split light in its different wavelengths. Here this information is used to determine the wavelengths of the Balmer-Series. With the formula

$$\frac{c}{\lambda} = \nu = R_H\left(\frac{1}{m^2} - \frac{1}{n^2}\right) \tag{1.1}$$

we caluclate the Rydberg-Frequency R_H. Here ν is the frequency of light and the electrone jumps from the n-th layer to the m-th layer.

All of these values are well known and the experiment is a learning excercice to get familier with spectroscopy.

2 Experiment

The experiment consists of first calibrating the spectrometer. Then to measure the hydrogene lines and calculate the Rydberg-Frequency R_H, the lattice-constant g, the limit of the Balmer-Series λ_{min} and the ionisation energy E_{ion} for a photon at the Rydberg-Frequency.

The equipment used is a spectrometer, helium lamp, hydrogenq lamp, a CDD sensor and a Laptop with the software "ThorImageCam". The spectrometer has a dial with a measurment scale of no unit. This value will be refered to as k-value k. It turns the mirror in the spectrometer which allows to see a different wavelength range on the sensor and screen. On the screen the picture is vied as 1-dimensional, since this spectrometer splits light in one dimension. The position on the screen will be referd to as pixel position p, where $0\ \text{px} \leq p < 1440\ \text{px}$. The unit of p is pixel px.

For calibration, the first step is to find the 0-th order. The corresponding k-value is k_0. From there the helium fuse is connected to the spectrometer. The helium lines all have known wavelengths [1, p. 149]. When a helium line is found, its k-value is noted down aswell as its wavelength in order to match a fit function later. After this step a k-value is fixed. For this fixed k-value we measured the pixel position of the visible helium lines. This is used to complete the fit function as a function of k-value and pixel position.

Then we conneced the helium lamp. After that we measered the k-values and pixel-position of the helium lines. In the next step the fit function can be applied to the k-values and pixel positition to find their wavelengths. From there the remaining values can be caluclated.

[1] Realtive deviation=$\frac{|3.07-3.29|}{3.29} \cdot 100\% \approx 6.69\%$

[2] $R_H = 3.29 \times 10^{15} Hz$ [1]

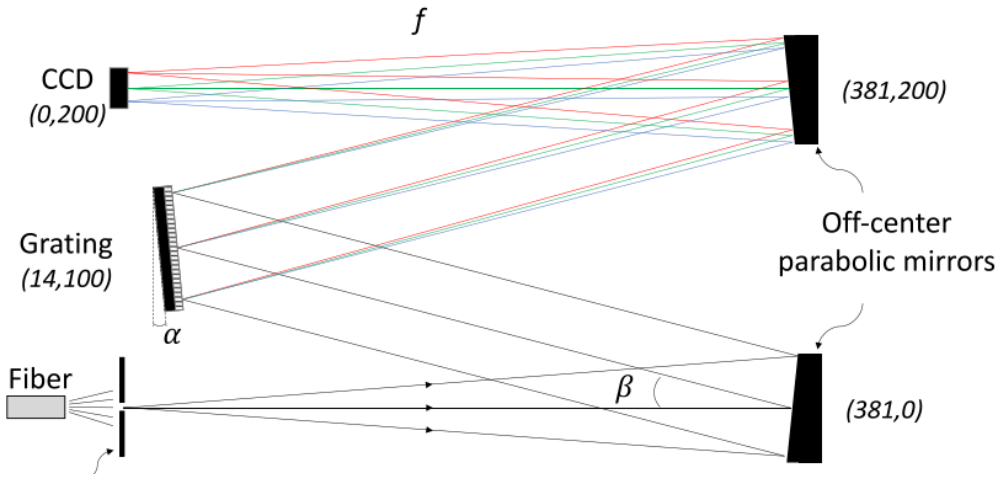


Figure 1: "(Color online) Schematic sketch of the Czerny–Turner spectrometer design. The spectrometer features an entrance slit, an off-center parabolic mirror that collimates the light, a ruled reflective grating that disperses the light, and a second off-center parabolic mirror that focuses the light on the CCD camera, which is then used to record the measured spectrum. The wavelength of lines visible in the spectrum is determined by the angle a of the grating that can be controlled by rotating a dial (not shown)."
Source: [1, p. 146]

3 Results

3.1

k_0 denotes the k-value which corresponds to the 0-th order. The measured value is $k_0 = 950$. This value is wrong. More details are in the discussion section.

The first measurement is to fit the wavelength of the center pixel position ($p_c = 720$ px) in dependance of dial position (k-value). The results are the following:

Wavelength λ [nm]	388.8	447.1	471.3	492.1	504.7	667.8	706.5
k-value	1310 ± 1	1131 ± 1	1101 ± 1	1072 ± 1	1070 ± 1	849 ± 1	800 ± 1

Table 1: Measured k-values for the helium spectral lines.
For uncertainties see Section A.2.i.

The first row contains the wavelength in nm of the measured light-strips at $p_c = 720$ px. The values are the literature values of the helium lamp [1, p. 149]. The second row contains the measurements. The measurement is the k-value. This is light from the helium lamp.

3.2

The second measurement is for a fixed k-value and measures data to fit the wavelength in dependance of pixel position (Figure 3a) the results are the following:

Wavelength λ [nm]	492.1	501.5	504.7
Pixel position [px]	475 ± 1	802 ± 1	920 ± 1

Table 2: Measured pixel positions of the helium spectral lines and their corresponding wavelengths.
For uncertainties see Section A.2.i.

The first row is again the known wavelengths of the measured light-stripes in nm of the helium lamp. The second row is the pixel position on the image. The image is 1440 px wide, therefore the pixel position p has $0 \text{ px} \leq p < 1440 \text{ px}$. We used the Jupyter Notbook from the Moodle to determine the pixel positions.

3.3

The third data-set is from the hydrogen lamp to measure the wavelengths of the Balmer-Series.

n	3	4	5	6
k-value	860 ± 1	1080 ± 1	1112 ± 1	1120 ± 1

Table 3: Measured k-values for the hydrogen spectral lines.
For uncertainties see Section A.2.i.

The first row is the Balmer-Series Equation (1.1). They are the corresponding measurements of the k-values.

4 Data Analysis

4.1

The fit function used is of the form

$$\lambda(p, k) = bk + a + \frac{p - p_{\text{cent}}}{d} \tag{4.1}$$

We used this function beacause our k_0-value was wrong. Therefore we now use three fit parameter (b, a, d) instead of two like in the manual [1, eq. 5]. With the first data set we obtained values for a and b. we used scipy.curve_fit for the regression. For the first data set we used the fit function $\lambda_c(k) = bk + a$ since $p = p_{\text{cent}}$. This gives the values $a = (1220 \pm 2)$ nm[3] and $b = (-0.663 \pm 0.002)$ nm[4]. Here we used the covariance matrix produced by curve_fit to determine the standart deviation for a and b. We used AI tools to assits debugging the Python program. All calculations are made in the Python program (see Section A.1).

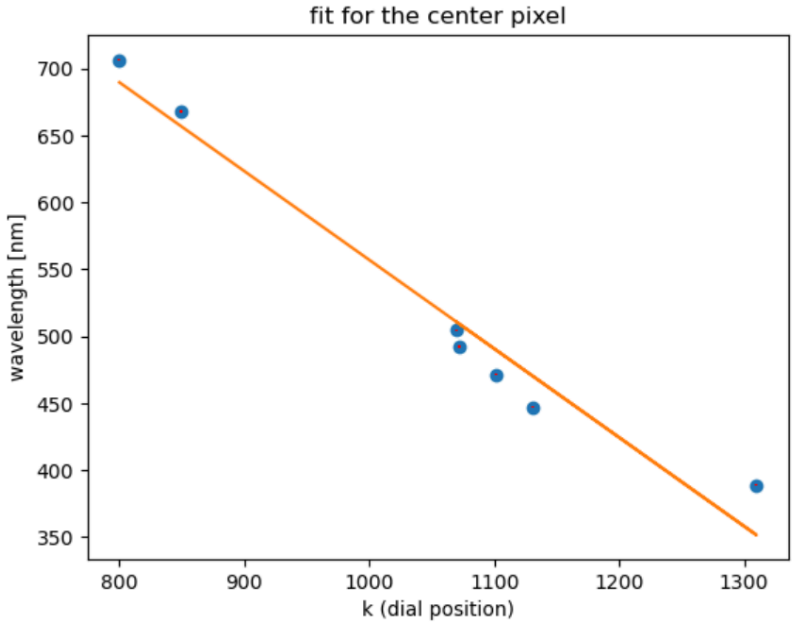


Figure 2: The Graphs shows the differents spactrallines of helium marked as points measured in the in the center pixel. On the x-axis we have the k-(dial position) and the y-axis is the wavelength in nanometers. The orange line is the least sqaures approximation of the points.

4.2

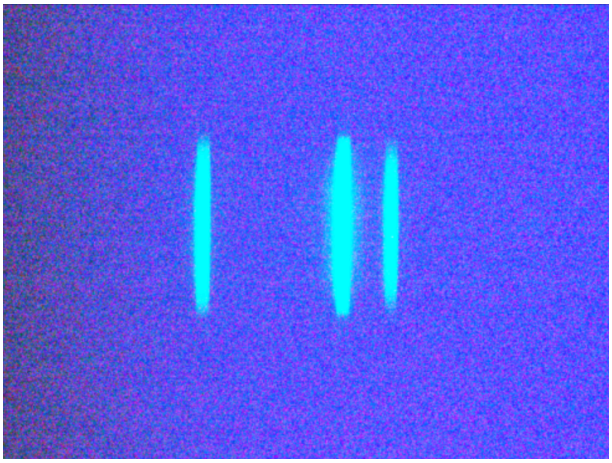
For the second data set the fit function is

$$\lambda(p, k_{\text{fix}}) = bk_{\text{fix}} + a + \frac{p - p_{\text{cent}}}{d} + e \tag{4.2}$$

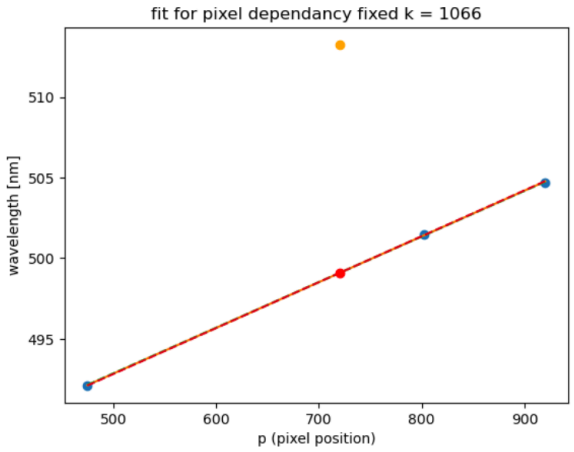
Without the e term this would be the final fit function but with a fixed $k = k_{\text{fix}}$ and a, b from Equation (4.1). We added e to compensate for an offset in the center pixel position. This is beacause the fit gives a fixed value for the center pixel. This fitted value is above all the points tho, which would lead to a worse fit here. This gives $d = (35.2 \pm$

[3] Section A.2.iii
[4] Section A.2.iii
[5] Section A.2.iii

$0.1)$ px nm^{-1}[5] and $e = (-14.17 \pm 0.02)\mathrm{nm}$[6]. We obtained the uncertanities from the curve fit covariance matrix. Again we used the help of AI-tools to help debug our Python-Program in Section A.1.



(a) Screenshot of the spectral range at k = 1066. Three spectral lines are visible.



(b) Pixel positions of the visible spectral lines. The blue dots represent the spectral lines. The red dot represents the fitted value of the center pixel plus e to compensate for the offset. The orange dot represents the fixed value for the center pixel without correction.

Figure 3: Spectral line detection and pixel position fitting.

4.3

From the fit we calculate the lattice constant g as $g = \frac{1}{d} = (-16.6 \pm 0.9) \cdot 10^{-5}\mathrm{nm}^{-1}$[7]. Where

$$d = \frac{\lambda_c}{2\cos(\beta)\sin(\alpha)}, \qquad [1] \tag{4.3}$$

with $\lambda_c = 667.8$ nm and

$$\beta = 15°, \qquad |\alpha| = (k - k_0)5.8 \times 10^{-5} = (3.29 \pm 0.02)° \qquad [1], \text{ Section A.2.ii} \tag{4.4}$$

k_0 is the k-value such that when $p = p_{\text{cent}} = 720$ px we have $\lambda(p_{\text{cent}}, k_0) = 0$.
Since

$$\frac{p - p_{\text{cent}}}{d} = 0 \Rightarrow \lambda(p_{\text{cent}}, k_0) = bk_0 + a = 0 \tag{4.5}$$

thus

$$k_0 = -\frac{a}{b} = 1840 \pm 5 \qquad \text{Section A.2.ii} \tag{4.6}$$

With the complete fit function Equation (4.1) we obtained the following wavelengths of the Balmer-Series through the fit function:

$$\lambda_3 = 650 \pm 2 \text{ nm}, \quad \lambda_4 = 504 \pm 2 \text{ nm}, \quad \lambda_5 = 483 \pm 2 \text{ nm}, \quad \lambda_6 = 477 \pm 2 \text{ nm} \qquad \text{Section A.2.iii} \tag{4.7}$$

With Equation (1.1) and the average of the wavelenths we can calculate the Rydberg-Frequenc. With using the relation:

$$R_H = R_\infty \cdot c \tag{4.8}$$

we get

$$R_H = (3.07 \pm 0.01)e15 \text{ Hz}, \qquad \text{Section A.2.ii} \tag{4.9}$$

Therefore with Equation (1.1) when $n \to \infty$

$$\lambda_{\min} = (391 \pm 2) \text{ nm}, \qquad \text{Section A.2.ii} \tag{4.10}$$

From the Rydberg-Frequency we continue to calculate the ionisation energy as

[6] Section A.2.iii
[7] Section A.2.ii

$$E_{\text{ion}} = \frac{hR_{\text{H}}}{e} = (12.7 \pm 0.1)\ \text{eV}, \qquad \text{Section A.2.ii} \tag{4.11}$$

where h is Plancks constant ($h \approx 6.63 \times 10^{-34} J \cdot s$) and e is the proton charge($e \approx 1.602 \times 10^{-19} C$).

To determine the standart deviations for the obtainded values. We used Gaussian-Error-Propagation. To debug our srcipt we used AI-tools to help.

5 Discussion

The k_0-value measured in the experiment is $k_0 = 950$. This measurment does not correspond to the actual k_0-vlaue but to the 587.5 nm wavelength light. This wavelength is yellow and was therefore misinterpreted as the 0-th order. In order to correct for this mistake, we used for the first fit function $\lambda_{\text{c}}\ (k) = bk + a$ instead of $\lambda_{\text{c}}\ (k) = b(k - k_0)$. The latter function is used in the manual [1].

The measurements of the hydrogene spectral lines have given:

Literature value [nm]	656	486	434	410
Experimental value [nm]	650 ± 2	504 ± 2	483 ± 2	477 ± 2

Table 4: Comparison of the experimental and literature wavelengths of the hydrogen spectral lines.
For uncertainties see Section A.2.ii

Also the experimental values from the Rydberg-Frequency, ionisation energy and the limit of the Balmer-Series λ_{min} when $n \to \infty$ deviate from the literature values [2].

	Rydberg frequency [Hz]	**Ionisation energy [eV]**	**Series limit [nm]**
Literature value	3.29×10^{15}	13.6	365
Experimental value	$(3.07 \pm 0.01) \times 10^{15}$	12.7 ± 0.1	391 ± 2

Table 5: Comparison of the experimentally determined Rydberg frequency, ionisation energy and series limit with the literature values.
For uncertainties see Section A.2.ii

The expected error traces back to the originally set values of $\Delta k = \pm 1$ and $\Delta p = \pm 1$ where Δk (Section A.2.i) is the error of the k-value and Δp the error in pixel position. However they do not account for the actual error. We therefore assume that there is some systematic error. As a main source we see errors in the measurements. This could have happened at different stages. Firstly in the calibration step with the helium lamp. A possible mistake is a misidentification of a helium line with another helium line and therefore a wrong data-point of the fit function. Another possible mistake is a wrong k-value. It is possible that some readings deviate by 100 beacause we found that the disply of the k-value was difficult to read for k-values near to a full hundreth. Also it is possible that there was a copying mistake whilst writing the values down. This all would lead to errors that propagate into the fit function. Since our data is visibly not linear we belive that one such mistake happend.

The error in the fit function is also visible in Figure 3b. The first fit when $p = p_{\text{cent}}$ in Equation (4.1) gives a fixed point for $p = p_{\text{cent}} = 720$ px and the free parameter varies the slope. However all data points are below the fixed point, likely beacause the first fit is already wrong by some offset at this position. This could lead to a wrong fit value, since the second fit (Equation (4.1) with $k = k_{\text{fix}} = 1066$) tries to compensate for the offset produced by the first fit.

Further mistakes could be wrong k-value or pixel position readings for the helium lines, which could also contribute to the error. Also the hydrogen lines with $n = 5$ and $n = 6$ were faint and could have easely been misread.

6 Conclusion

In this experiment we calibrated a spectrometer and measured the hydrogene lines of an hydrogene lamp. These results were used to calculate the Rydberg-Frequency R_H, the lattice-constant g, the limit of the Balmer-Series λ_{min} and the ionisation energy E_{ion} for a photon at the Rydberg-Frequency.

The experimental values of R_H, λ_{min} and E_{ion}, lie within 10% of the literature values. The error is not covered by the standart deviation of the obtained values. Since the fit data differs from the linear fit model, we concluded that there is systematic error in the data. We see misidentification of helium lines in the calibration step and w

A Appendix

A.1 python skript

```
# %%
import numpy as np
import matplotlib.pyplot as plt

# %%
p_cent = 720                                                    # image is 1440 wide
k0 = 950                                                        # +-1 measured value
k_cent = np.array([1070, 1072, 1101, 1131, 1310, 849, 800]) # +-1 measured values of k
lamda_cent = np.array([504.7, 492.1, 471.3, 447.1, 388.8, 667.8, 706.5])
                                                                # corresponding wavelength values from
manual

# %%
# fit model for wavelength depending on dial position k
def lamda_cent_fit (k, bb, aa):
    return bb * k + aa

# %%
std_k = 1
std_k_cent = np.ones(7) * std_k # uncertenties of k values

from scipy.optimize import curve_fit
# first fit to convert x-error to y-error
popt, pcov = curve_fit(lamda_cent_fit, k_cent, lamda_cent, absolute_sigma=True)

b_prov = popt[0]

# y-error
std_lamda_from_k = np.abs(b_prov * std_k_cent)

# actual fit
popt, pcov = curve_fit(lamda_cent_fit, k_cent, lamda_cent, sigma=std_lamda_from_k,
absolute_sigma=True)

# fit parameters a, b for lamda(k) = b*k + a
b = popt[0]
a = popt[1]
print(a)
print(b)

print('========')

std_b = np.sqrt(pcov[0, 0])
std_a = np.sqrt(pcov[1, 1])
print(std_a)
print(std_b)

plt.plot(k_cent, lamda_cent, linestyle='', marker='o')
plt.plot(k_cent, lamda_cent_fit(k_cent, b, a))
plt.xlabel('k (dial position)')
plt.ylabel('wavelength [nm]')
plt.title('fit for the center pixel')
plt.errorbar(k_cent, lamda_cent, std_lamda_from_k, linestyle = '', color='r')


# %%
k_fix = 1066                              # +-1 measured value
p_pos = np.array([475, 802, 920])         # pixel position from image +-1
lamda_k = np.array([492.1, 501.5, 504.7]) # values from manual (wavelength)
```

```
# %%
# fit model including pixel position
def lamda_fit (p, dd, ee):
    return b * k_fix + a + (p - p_cent) / dd + ee

# %%
std_p = 1

from scipy.optimize import curve_fit
# first fit to determine y-error from x-error
popt, pcov = curve_fit(lamda_fit, p_pos, lamda_k, absolute_sigma=True)

d_prov = popt[0]
std_lamda_from_p = np.abs(std_p / d_prov)

# actual fit

<zweites>
popt, pcov = curve_fit(lamda_fit, p_pos, lamda_k, sigma=std_lamda_from_p, absolute_sigma=True)
d = popt[0]
e = popt[1]

print(d)
print(e)
std_d = np.sqrt(pcov[0, 0])
print(std_d)
std_e = np.sqrt(pcov[1, 1])
print(std_e)

# %%
plt.plot(p_pos, lamda_k, linestyle='', marker='o')
offset = 0
plt.plot(p_pos, lamda_fit(p_pos, d, e) - offset)
plt.plot(p_pos, lamda_fit(p_pos, d + std_d, e) - offset, linestyle='--')
plt.plot(p_pos, lamda_fit(p_pos, d - std_d, e) - offset, linestyle='--')

plt.xlabel('p (pixel position)')
plt.ylabel('wavelength [nm]')
plt.title('fit for pixel dependancy fixed k = 1066')

plt.plot(p_cent, lamda_fit(p_cent, d, e) - offset, marker='o', color='r')
plt.plot(p_cent, lamda_fit(p_cent, d, 0) - offset, marker='o', color='orange')

# %%
# final fit function
def lamda (p, k):
    return b * k + a + (p - p_cent) / d



# %%
# calculation of the lattice constant

# %%
lamda_at_center = 667.8                 # wavelength of middle pixel
beta = 15 * 2 * np.pi / 360             # 15° in radians
k_pos = 850                             # +-1 dial position
k0_fit = (-1) * a / b                   # k0 from the fit
alpha = (k_pos - k0_fit) * 5.8 / 10**5  # calculates the angle alpha based on manual
print(alpha / (2 * np.pi) * 360)        # prints alpha in degrees
print(k0_fit)
print(lamda(p_cent, k0_fit))
```

```
# %%
# lattice constant:
d = lamda_at_center / (2 * np.cos(beta) * np.sin(alpha))
print(1 / d)
print(10**5 / d)

# %%
# error prop:
del_k0_fit_a = (-1) / b
del_k0_fit_b = a / b**2
std_k0_fit = np.sqrt((del_k0_fit_a * std_a)**2 + (del_k0_fit_b * std_b)**2)


del_alpha_k_pos = 5.8 / 10**5
del_alpha_k0_fit = (-1) * 5.8 / 10**5
std_alpha = np.sqrt((del_alpha_k_pos * std_k)**2 + (del_alpha_k0_fit * std_k0_fit)**2)

del_g = 2 * np.cos(beta) * np.cos(alpha) / lamda_at_center
std_g = np.sqrt((del_g * std_alpha)**2)

print(std_g)
print("====")
print(std_k0_fit)
print(std_alpha * 180 / np.pi)


# %%
measurements = np.array([860, 1080, 1112, 1120])    # +-1 k values
balmer_lamda = lamda(p_cent, measurements)          #literature 656.279, 486.135, 434.0472, 410.1734
print(balmer_lamda)
c = 299792458

print('==============')

# error porp:
del_f_a = 1                             # = 1
del_f_b = measurements                  # = k
del_f_d = -(p_cent - p_cent) / d**2     # = -(p - p_cent) / d^2
del_f_p = 1 / d                         # = 1 / d
del_f_k = b                             # = b
std_lamda = np.sqrt((del_f_a * std_a)**2
               + (del_f_b * std_b)**2
               + (del_f_d * std_d)**2
               + (del_f_p * std_p)**2
               + (del_f_k * std_k)**2)

print(std_lamda)

# %%
# calculated values for rydborg frequency
nn = np.array([3, 4, 5, 6])                                 # balmer series
balmer_lamda_m = balmer_lamda / 10**9                       # wavelength in m
ryd_freq = c / balmer_lamda_m * 1 / ((1 / 4) - (1 / nn**2))   # formula from manual
print(ryd_freq)
print(np.mean(ryd_freq))
print(np.mean(ryd_freq) / 10**15)
print('---')
print(np.mean(ryd_freq) / c)

print('==============')

# error prop:
del_ryd_lamda = (-1) * c / (balmer_lamda_m)**2 * 1 / ((1 / 4) - (1 / nn**2))
```

```
std_ryd = np.sqrt((del_ryd_lamda * std_lamda / 10**9)**2)
print(std_ryd)
print(np.mean(std_ryd) / 10**15)

# %%
# limit of series: f = Rh / 4 | lamda = 4 * c / Rh
print(c / np.mean(ryd_freq) * 4 * 10**9)

print('============')
# error prop:
del_lamda_lim = (-1) * 4 * c / np.mean(ryd_freq)**2
std_lamda_lim = np.sqrt((del_lamda_lim * np.mean(std_ryd))**2)
print(std_lamda_lim * 10**9)

# %%
# E_photon = h * c / lamda          (h is planks constant)
# E_photon[eV] = E_photon / e       (e is electornic charge)
e = 1.602176634 * 10**(-19)
h = 6.62607015 * 10**(-34)

print(h * np.mean(ryd_freq) / e)

print('=======')
# error prop:
del_E_photon = h / e
sig_E_photon = np.sqrt((del_E_photon * np.mean(std_ryd))**2)
print(sig_E_photon)
```

A.2 Uncertainties

All the uncertainties were calculatet via a python script. For more detail see Section A.1.

A.2.i Uncertainties of measured values

The uncertany of an measured quantity is the smalest value we can the measureing tool can distinguish.
Ex. if we use a ruler which has a smallest quantity 'mm' we get 'something' $\pm$ 1mm

A.2.ii Gaussian Error Propagation

For a function $f(x_1, x_2, ...)$ with independent uncertainties yields the following total uncertainty

$$\sigma_f = \sqrt{\sum_{i=1}^{n} \left(\frac{\partial}{\partial x_i} f \cdot \sigma_{x_i} \right)^2} \tag{1.1}$$

where σ_{x_i} is the uncertantie of corresponding parameter and $\partial_{x_i} f$ is the dericitive of the function f respect to x_i.
As an example. If we have the function

$$f(\alpha, \beta) = \sin(\alpha) \cdot \sin(\beta) \tag{1.2}$$

Where $\sigma_\alpha \neq 0$ and $\sigma_\beta = 0$ wich means that α could have been a measurement and β a given known angle without uncertainty the total uncertainty would be:

$$\sigma_f = \sqrt{(\cos(\alpha)\sin(\beta)\sigma_\alpha)^2} = |\cos(\alpha)\sin(\beta)\sigma_\alpha| \tag{1.3}$$

Conclusion: Gaussian error propagation is used to estimate the uncertainty of a calculated quantity based on the uncertainties of the variables used to calculate it.

A.2.iii Uncertainties from `curve_fit`

The function `curve_fit` from `scipy.optimize` fits a function to measured data and returns the optimal parameters (`popt`) and their covariance matrix (`pcov`).

The uncertainties of the fitted parameters are calculated from the diagonal elements of the covariance matrix:

$$\sigma_{p_i} = \sqrt{\mathrm{pcov}_{ii}} \tag{1.4}$$

The parameter `absolute_sigma` determines how the provided measurement uncertainties are interpreted. If `True`, they are treated as absolute uncertainties. If 'False', the covariance matrix is scaled based on the residuals of the fit, while still accounting for the relative measurement uncertainties. In both cases, `curve_fit` automatically estimates the uncertainties of the fitted parameters.

Bibliography

[1] A. Eggenberger, T. Smolenski, and M. Kroner, "A simple state-of-the-art spectrometer for student labs: Cost-efficient, instructive, and widely applicable," *American Journal of Physics*, vol. 92, no. 2, 2024, doi: 10.1119/5.0164044.

[2] Wikipedia, "Rydberg-Konstante." Accessed: Oct. 30, 2026. [Online]. Available: https://de.wikipedia.org/wiki/Rydberg-Konstante#Rydberg-Frequenz_und_Rydberg-Energie

Corrections

- We made some general corrections, such as fixing spelling mistakes and clarifying unclear phrases.
- We improved the fit in Figure 3(b) without forcing it through the incorrect centre value of 513.3 nm.
- We combined the previous Figures 3 and 4 into Figure 3(a) and Figure 3(b).
- For the uncertainties, we described the general cases in Appendix A.2 and specified in the main text which method was used in each case. The detailed calculations can be found in the Python script. Is this an appropriate way to present the uncertainty analysis?
- Error bars are already included in Figures 2 and 4, but they are barely visible because the uncertainties are only ±1 k. We are also not entirely sure whether we used the correct uncertainties.